

General Residential Zone (GRZ) Land Development Capacity

Technical Documentation

1. Overview

This SQL script implements a **land development capacity (LDCAP) model** for GRZ (**General Residential Zone**) properties.

Purpose

- Identify **vacant, infill, and fully developed properties**
- Estimate **development potential (PHU yield / lots)**
- Consider:
 - Planning overlays
 - Hazards
 - Building footprints
 - Access (road frontage)
 - Geometric constraints

Core capabilities:

- Spatial constraint analysis
- Buildable land estimation
- Development classification
- PHU yield modelling
- Aggregation for reporting

System Architecture:

The system is structured as a **modular, materialized view pipeline**:

```
1 INPUT DATA
2   ↓
3 Overlay Analysis (core engine)
4   ↓
5 Building Analysis
6   ↓
7 Viability Testing (vacant / infill / developed)
8   ↓
9 Viable Outputs
```

10	↓
11	Aggregation

Each stage is implemented as a **stored procedure that creates a materialized view.**

2. Master Orchestration Procedure

`property.create_mv_property_grz_ldcap()`

This is the **entry point** and controls full pipeline execution.

What it does:

1. Drops all dependent materialized views
2. Rebuilds them in correct dependency order

Execution order:

1	building_quadrant_stats
2	→ building_quadrant_summary
3	→ viability_overlay_analysis
4	→ infill_viability_testing
5	→ vacant_viability_testing
6	→ fully_developed_testing
7	→ infill_viable
8	→ vacant_viable
9	→ fully_developed
10	→ summary
11	→ summary_suburbs

Key insight

This ensures:

- All downstream tables always use **fresh upstream outputs**
 - Dependencies are correctly sequenced
-

3. Core Analytical Components

3.1 Building Quadrant Analysis

building_quadrant_stats

Splits each property into **4 quadrants (NE, NW, SE, SW)**.

Process:

1. Select GRZ properties
2. Compute:
 - centroid
 - bounding box
3. Create vertical and horizontal split lines
4. Split polygon into 4 quadrants
5. Assign quadrant IDs (1–4)

Adds:

- **Frontage detection**
 - Based on intersection with road frontage
- **Building overlap**
 - Computes how buildings intersect each quadrant
- **Buildable quadrant**
 - TRUE if:
 - no buildings
 - frontage exists

Output includes:

- quadrant_geom
 - frontage_length
 - quadrant_is_front_facing
 - overlap_area, building_area
 - is_buildable_quadrant
-

3.2 Quadrant Summary

building_quadrant_summary

Aggregates per property:

- Total buildings
- Total building area
- Building count per quadrant (NE/NW/SE/SW)
- Number of buildable quadrants
- Front-facing quadrant
- Total frontage

Adds:

- **property geometry**
-

4. Viability Overlay Analysis (Core Engine)

4.1 Procedure

`property_grz_ldcap_viability_overlay_analysis`

This is the **central analytical engine**.

What it calculates:

1. Property metrics

- Area
- Road frontage
- Vacancy flag

2. Overlay constraints (via Configuration table and Overlay Stats function)

Each return:

- area
- percentage overlap

3. Combined constraint geometry

```
1 overlay_geom = UNION of all constraints
```

4. Developable land

```
1 developable_geom = property - overlay_geom
```

Also calculates:

- developable_area
- developable_percentage

4.2 Configuration Table

property_grz_ldcap_overlay_analysis_config

Defines all constraint sources.

Columns:

Column	Purpose
alias	output column name
schema_name	source schema
table_name	source table
filter_sql	optional filter
include_in_overlay	include in constraint geometry
sort_order	column ordering

4.3 Dynamic Column Generation

Columns are generated dynamically:

```
1 prop_overlap_area_<alias>
2 prop_overlap_pct_<alias>
```

Using:

```
1 MAX(...) FILTER (WHERE alias = ...)
```

4.4 Single-Pass Overlay Evaluation

All overlays processed in one step:

```
1 FOR EACH property:
2     LOOP config rows
3     CALL calc_overlap_stats_record()
```

Benefits:

- Eliminates repeated spatial intersections
 - Reduces runtime significantly
 - Ensures consistency
-

4.5 Overlay Geometry

```
1 overlay_geom =  
2     ST_UnaryUnion(collect(all selected overlap geometries))
```

Represents:

- ✓ total constrained land

4.6 Developable Land

```
1 developable_geom = property_geom - overlay_geom
```

Metrics:

- developable_area
- developable_pct

4.7 Additional Metrics

- Building footprint overlap
 - Road frontage
 - Vacancy flag
-

5. Overlap Stats Function

`general.calc_overlap_stats_record(...)`

Purpose

Reusable function to:

- intersect property with any dataset
- return:
 - overlap geometry
 - overlap area
 - overlap %

Key features:

- dynamic SQL
- geometry validation
- safe handling of empty cases
- supports filtering (where_clause)

6. Viability Pipelines

6.1 Vacant Pipeline

Step 1: vacant_viability_testing

Filters properties by excluding:

- facilities / open space
- small lots (<400 or <800 sqm overlay logic)
- overlapping properties
- known bad geometries
- recent building consents

Then:

- removes overlay constraints
- runs **maximum inscribed circle (MIC)** to test building platform viability

Criteria:

- radius $\geq 7\text{m}$
 - sufficient area after constraints
-

Step 2: vacant_viable

Calculates:

Potential lots:

1	<code>floor(developable_area / 400)</code>
2	<code>or /800 if overlay applies</code>

PHU yield logic:

- 1 lot $\rightarrow$ 1
- 2–6 $\rightarrow$ same
- ≥ 7 :
 - frontage $\geq 10\text{m}$ $\rightarrow$ 70% yield
 - else $\rightarrow$ capped at 6

Access constraint:

- frontage $\geq 3.6\text{m}$ minimum
-

6.2 Infill Pipeline

Step 1: infill_viability_testing

More complex than vacant.

Additional filtering:

- building coverage $\geq 25\%$
- multiple buildings
- setbacks
- access width ($\geq 3.6\text{m}$)
- centroid conflicts

Geometry processing:

1. remove overlays
2. remove buildings (+ 3m buffer on existing building)
3. check remaining usable land

Advanced checks:

- setback clearance
- access side count
- building quadrant logic

Platform test:

- MIC radius $\geq 7\text{m}$
 - minimum usable space
-

Step 2: infill_viable

Same structure as vacant but:

PHU yield differs:

- 1 lot $\rightarrow 0$ (no gain)
- 2–6 $\rightarrow$ lots - 1
- ≥ 7 :

- frontage $\geq 10\text{m}$ $\rightarrow$ reduced (70%)
 - else $\rightarrow$ capped at 6
-

6.3 Fully Developed

fully_developed

Defined as:

- GRZ properties not:
 - vacant
 - infill viable
 - excluded overlays

Output:

- 1 lot only
 - 0 PHU yield
-

7. Summary Outputs

7.1 Global Summary

property_grz_ldcap_summary

Aggregates:

- number of properties
- total lots
- total PHU
- total area

By:

- development type:
 - Vacant / Infill
- scale:
 - Single
 - Small (2–6)
 - Large (7+)

7.2 Suburb Summary

property_grz_ldcap_summary_suburbs

Same as above but grouped by:

- suburb (SA2)
-

8. Exclusion Threshold Framework

Table:

property_grz_ldcap_exclusion_thresholds

Purpose:

Data-driven rule system for excluding properties.

Supports:

- operators: > >= < <= = BETWEEN
- linked to:
 - overlay percentages

Example rules:

- slope > 30%
- flood ≥ 30%
- liquefaction ≥ 30%

Used in:

- vacant_viable
 - infill_viable
-

9. Building Ranking Function

property.property_buildings_ranking(prop_no)

Ranks buildings by likelihood of being the **primary dwelling**.

Criteria:

- area (largest better)
- distance to frontage (closer better)
- alignment with frontage
- back-of-lot penalty
- heuristic classification

Output:

- ranked buildings
 - or fallback point if none exist
-

10. Key Design Patterns

10.1 Materialized View Pipeline

- All outputs cached
- Full refresh model

10.2 Progressive Filtering

Each stage:

1 raw → testing → viable → summary

10.3 Geometry-first logic

Everything derived from:

- spatial intersection
- buffering
- difference
- MIC

10.4 Strong performance design

- Extensive GIST indexes
- lateral joins to avoid recomputation
- shared overlay function

10.5 Config-driven rules

- thresholds table enables tuning without SQL changes

10.6 Incremental refinement

- 1 Raw property data
- 2 → Apply exclusions
- 3 → Apply overlays
- 4 → Remove buildings
- 5 → Test buildable space
- 6 → Calculate yield

10.7 Real-world planning heuristics

Includes:

- Frontage constraints
- Minimum lot size
- Building separation
- Access requirements

11. Data Outputs (Core Tables)

Table	Purpose
..._overlay_analysis	Core constraint + developable land
..._vacant_viable	Final vacant sites
..._infill_viable	Final infill sites
..._fully_developed	Non-developable
..._summary	Aggregated totals
..._summary_suburbs	Suburb rollup
..._building_quadrant_stats	Fine-grained building layout
..._building_quadrant_summary	Property-level building stats

12. End-to-End Conceptual Flow

- 1 GRZ properties
- 2 ↓
- 3 Overlay analysis (constraints + developable land)
- 4 ↓
- 5 Branch pipelines:

6	└─ Vacant
7	└─ Infill
8	└─ Fully developed
9	↓
10	Viability filtering
11	↓
12	Lot estimation + PHU yield
13	↓
14	Aggregation (global + suburb)

Final Summary

This script is a **full spatial decision engine** for residential capacity that:

- Combines **planning + hazard + geometry + access**
 - Uses **robust spatial modelling (PostGIS-heavy)**
 - Uses **dynamic configuration** for overlays
 - Applies **clear staged filtering**
 - Produces **policy-ready outputs (PHU, development categories)**
-